

emdash — Functorial programming for ω -categories in Lambdapi (v3.2 arrow induction, profunctors, and weighted limits)

<https://github.com/hotdocx/emdash>

Abstract

We present the v3.2 architecture of **emdash**, a Lambdapi specification for a programming language and prototype proof assistant for strict/lax higher ω -categorical structure. The development is fully internalized in dependent type theory and computational in the proof-theoretic sense: structural categorical equalities are oriented as normalization, or cut-elimination, steps.

The basic construction is the directed dependent hom. For a category-valued family $E : K \vdash \mathbf{Cat}$, where $\vdash$ denotes a functor category, and fixed data $x : K$ and $u \in E[x]$, emdash forms a functorial object

$$\mathrm{homd}_E(x, u) : \prod_{y:K^{\mathrm{op}}} (E[y^-] \vdash_{[y]} (\mathrm{Hom}_K(x, y)^{\mathrm{op}} \vdash \mathbf{Cat}))$$

Here $\vdash_{[y]}$ is the mixed-variance displayed version of $\vdash$, and y^- marks that the E -argument occurs contravariantly. At a fixed y , its value at $v \in E[y]$ and $f : x \rightarrow y$ is

$$\mathrm{homd}_E(x, u)[y][v][f] = \mathrm{Hom}_{E[y]}(E[f](u), v).$$

This is the construction that organizes arrows in Sigma totals:

$$\mathrm{Hom}_{\Sigma E}((x, u), (y, v)) = \Sigma_{f:x \rightarrow y} \mathrm{Hom}_{E[y]}(E[f](u), v).$$

The same normalization-first architecture drives this simplicial ω -iteration and also covers product/curry structure, computational adjunctions, structural operations such as weakening/symmetry/contraction, and ordinary hom-action cut elimination. In the current v3.2 kernel it also supports a symbolic but checked calculus of Cat-valued profunctors: representables, reindexing, tensor, co-Yoneda maps, covariant and contravariant internal homs, weighted limits as representability, preservation of weighted limits by right adjoints, the dual preservation theorem for left adjoints and weighted colimits, and a primitive directed-inductive join category.

The motivating example is the familiar shape of path induction in dependent type theory, now directed. For a category Z and an object $x : Z$, emdash uses the outgoing-arrow category, i.e. the coslice/undercategory

$$\mathrm{PathOut}_Z(x) := x \downarrow Z := \Sigma_{y:Z} \mathrm{Hom}_Z(x, y).$$

The object $\iota_x = (x, \mathrm{id}_x)$ is initial in $\mathrm{PathOut}_Z(x)$. For $a = (y, p)$, the canonical arrow $\iota_x \rightarrow a$ is the arrow p itself; when a reusable name is needed, we write it as

$$\rho_a^x : \iota_x \rightarrow a.$$

Thus, for a motive $E : \mathbf{PathOut}_Z(x) \vdash \mathbf{Cat}$ and $u \in E(\iota_x)$, fixed-source directed induction gives the section

$$\mathbf{Ind}_x(E, u) : \prod_{a:\mathbf{PathOut}_Z(x)} E(a), \quad \mathbf{Ind}_x(E, u)(a) = E(\rho_a^x)(u).$$

Write $\mathbf{Rep}_Z(t)$ for the covariant representable $\mathbf{Hom}_Z(t, -)$. For the composition motive

$$E[(y, p)] := \mathbf{Rep}_Z(y) \vdash \mathbf{Rep}_Z(x),$$

with initial datum $\mathbf{id} : \mathbf{Rep}_Z(x) \vdash \mathbf{Rep}_Z(x)$, this computes to ordinary composition:

$$\mathbf{Ind}_x(E, \mathbf{id})[(y, p)][z][q] \rightsquigarrow q \circ p.$$

The latest runtime owner for this benchmark is the hom-action normal form `hom_postcomp_fapp0(id_Z, q, p)`. The ordinary composite `comp_fapp0(q, p)` is still available as the typed categorical view; the distinction is one example of the v3.2 boundary between runtime cut-elimination heads and proof-time convertibility surfaces.

The new phenomenon appears when the source object x itself is internalized. For an arrow $r : x \rightarrow y$, precomposition gives

$$\mathbf{PathOut}_Z(r) : \mathbf{PathOut}_Z(y) \vdash \mathbf{PathOut}_Z(x), \quad \mathbf{PathOut}_Z(r)(z, q : y \rightarrow z) = (z, q \circ r).$$

Once induction is internalized as a construction varying in x , the target section-taking construction

$$x \mapsto \left(E \mapsto \prod_{a:\mathbf{PathOut}_Z(x)} E(a) \right)$$

is itself displayed over the moving source object. Its transport/comparison along r is not the identity; it is the section-pullback functor

$$\prod_{a:\mathbf{PathOut}_Z(x)} E(a) \vdash \prod_{b:\mathbf{PathOut}_Z(y)} E(\mathbf{PathOut}_Z(r)(b)), \quad s \mapsto (b \mapsto s(\mathbf{PathOut}_Z(r)(b))).$$

This is the lax naturality/functoriality layer exposed by the internalized formulation of directed path induction; the `Lambdapi` kernel packages the source-indexed theorem as `PathInd_transfd(Z)`.

The profunctor part of the development follows the same rule. A weighted-limit theorem is not represented as a single propositional statement detached from computation. It is a comparison object whose push/pull operations act on arbitrary incoming profunctor maps and cancel by normalization. This is how the formalization proves that right adjoints preserve $\mathbf{Cat}$ -valued weighted limits, and how the dual left-adjoint colimit theorem is obtained by opposite normalization.

Contents

- Abstract: summary of the v3.2 architecture and main checked computations.
- 1. Introduction.
- 1.1 What v3.2 Contributes.

- 1.2 Road Map.
- 1.3 What "Checked" Means.
- 2. A Type-Theoretic Foundation For Directed Families.
- 3. Dependent Hom, Sigma Totals, And Sections.
- 3.1 Sigma Totals.
- 3.2 Pi Sections.
- 4. Outgoing Arrows And The Canonical `rho` Arrow.
- 5. Arrow Induction.
- 5.1 The Telescope Theorem.
- 5.2 The Derived Sigma-Total Presentation.
- 6. Strictness, Laxity, And Directed Induction.
- 7. Composition As The Main Computation.
- 8. Surface Syntax And Kernel Names.
- 9. Computational Method And Checked Evidence.
- 10. Cat-Valued Profunctors And Representables.
- 11. Tensor, Co-Yoneda, And Internal Hom.
- 12. Weighted Limits, Adjunctions, And Duality.
- 13. Directed-Inductive Join Categories.
- 14. Equality, Deflso, And Normalization Boundaries.
- 15. Conclusion.
- Appendix A. Identifier Glossary.

1. Introduction

Higher category theory is hard to mechanize because coherence is not a small side condition. Every definition carries functoriality, naturality, and higher-dimensional compatibility data. In ordinary mathematical prose, much of this data disappears into phrases such as "by functoriality" or "by naturality." In a proof assistant, those phrases either become explicit proof terms or they must compute.

The emdash design chooses the second option whenever the equation is structural enough to be part of the computational theory. Categories, functors, transformations, category-valued families, dependent sums, dependent products, and dependent homs are internal objects of the `Lambdapi` specification. Coherence facts are oriented as rewrite and unification rules with stable normal forms.

This paper explains the v3.2 development for a reader who knows dependent type theory or HoTT, but not the emdash codebase. The central claim is:

category-theoretic arrow induction can be expressed as a computational theorem.

The word "arrow" is important. The theorem is not HoTT identity elimination for paths in a type. It is an induction principle for outgoing directed arrows inside a category. The construction resembles path induction because every outgoing arrow $p : x \rightarrow y$ has a canonical comparison from the reflexive outgoing arrow (x, id_x) to (y, p) , but that comparison lives in a Sigma category of arrows, not in an identity type.

1.1 What v3.2 Contributes

The v3.2 article focuses on eight checked contributions.

1. **Directed dependent homs.** Homs in total categories and section actions are organized by one dependent-hom construction over a base arrow.
2. **Outgoing-arrow categories.** For each object $x : Z$, $\text{PathOut}_Z(x)$ is the Sigma total of the representable family $\text{Hom}_Z(x, -)$.
3. **Synthetic arrow induction.** For $x : ^n Z ; E : ^n (\text{PathOut}_Z(x) \vdash \text{Cat})$, the checked construction has type $E[(x, \text{id}_x)] \vdash \prod (q : ^n \text{PathOut}_Z(x)), E[q]$; the kernel packages the source-indexed telescope as $\text{PathInd_transfd}(Z)$.
4. **Strict and lax directed transport.** General displayed functors carry computable transport-comparison cells over base arrows; strict or cartesian constructors, including representable precomposition, collapse those cells to canonical identities.
5. **Computational composition.** The transitivity benchmark reduces a fully expanded path-induction expression to $q \circ p$. Runtime computation exposes the reusable hom-action owner $\text{hom_postcomp_fapp0}(\text{id}_Z, q, p)$, while a typed proof-time view records the ordinary composite $\text{comp_fapp0}(q, p)$.
6. **Cat-valued profunctors.** The kernel has a checked facade for $\text{Prof}(A, B) = A^{\text{op}} \times B \vdash \text{Cat}$, representables, reindexing, shaped cells, and shaped elements.
7. **Tensor, co-Yoneda, and internal hom.** A symbolic tensor and fixed covariant/contravariant implication cores support eval/lambda cancellation, fixed co-Yoneda beta/fusion, and functorial fixed-endpoint tensor action.
8. **Weighted limits, duality, and directed induction.** Weighted limits are implemented as computational profunctor representability; right adjoints preserve them by composing comparisons, left adjoints preserve the dual weighted colimits by opposite normalization, and Join_cat checks a primitive directed-inductive recursor with an internally natural cross cell.

1.2 Road Map

The paper has two connected layers. The first layer, Sections 2-9, develops the directed-family calculus far enough to state and check arrow induction. Section 2 fixes the basic directed type-theoretic reading of categories, functors, category-valued families, and section categories. Section 3 explains dependent homs, Sigma totals, and Pi sections, which are the structural ingredients used by outgoing-arrow categories. Section 4 defines $\text{PathOut}_Z(x)$ and the canonical rho arrow. Section 5 packages fixed-source, source-indexed, and Sigma-total arrow induction. Section 6 explains where strict and lax transport enter the theorem. Section 7 gives the main normalization benchmark: applying arrow induction to the composition motive computes to categorical composition. Sections 8 and 9 then record the surface syntax and the kernel-evidence discipline used by the checks.

The second layer, Sections 10-14, shows that the same cut-elimination discipline scales beyond the PathOut theorem. Section 10 introduces the Cat-valued profunctor facade, representables, reindexing, and shaped cells. Section 11 adds the symbolic tensor, fixed co-Yoneda maps, and covariant and contravariant internal homs. Section 12 formulates weighted limits as profunctor representability, proves right-adjoint preservation by computation, and obtains the dual left-adjoint colimit theorem by opposite normalization. Section 13 records the primitive directed-inductive join category. Section 14 separates equality, DefIso , profunctor comparison, and the runtime/proof-time normalization boundary.

Section 15 concludes. Appendix A collects kernel identifier correspondences. The essential artifact and validation facts are part of the computational method discussion in Section 9, where they support the checked-evidence claims rather than forming a separate implementation section.

1.3 What "Checked" Means

The paper uses checked in the Lambdapi sense. An assertion

```
Γ ⊢ left ≡ right
```

is accepted only when `left` and `right` are convertible under beta-reduction, definitions, rewrite rules, and unification rules. For the arrow-induction story, the checked evidence has three layers.

1. The theorem interfaces `typecheck`, including the unfolded telescope `E[(x,id_x)] ⊢ Π (q : ^n PathOut_Z(x)), E[q]`.
2. Their projections compute; for example, the source-indexed theorem reduces at source `x` to the fixed-source construction introduced in Section 5.
3. Fully expanded applications normalize to ordinary categorical terms, for example the composition benchmark reduces to $q \circ p$. The runtime normal form is the hom-action cut `hom_postcomp_fapp0 Z Z (id_func Z) x y z q p`; the typed ordinary composition view remains `comp_fapp0 Z x y z q p`.

Thus the paper is not claiming only that the theorem is mathematically plausible. The theorem surface, its projections, and the benchmark computation are part of the same executable rewrite system.

We follow a formula-first convention for implementation names. When the paper mentions a Lambdapi identifier such as `Sigma_cat` or `PathInd_transfd`, the surrounding display gives the corresponding mathematical or emdash surface notation first; Appendix A collects the identifier correspondences.

2. A Type-Theoretic Foundation For Directed Families

The surface notation is intentionally close to dependent type theory, but the base of a family is directed. A category K has real arrows, not just points. Consequently, a family over K must explain how fibre objects move along base arrows.

The core reading is:

<code>A : Cat</code>	a category
<code>x : A</code>	an object, formally $x : \text{Obj}(A)$
<code>a →^A b</code>	hom-category $\text{Hom}_A(a,b)$
<code>F : A ⊢ B</code>	ordinary functor
<code>F[x]</code>	object action
<code>F[f]</code>	arrow action
<code>E : K ⊢ Cat</code>	category-valued family over K
<code>E[k]</code>	fibre category at k
<code>E[f](u)</code>	transport of u along $f : k \rightarrow k'$

The hom between two objects is itself a category. A 1-cell $f : x \rightarrow y$ is an object of $\text{Hom}_A(x,y)$; a 2-cell is an arrow in that hom-category; higher cells are obtained by iterating the same pattern. This is the ω -oriented part of the foundation.

The universe of categories is also a category:

```
Obj(Cat)      = categories
Hom_Cat(A,B) = A ⊢ B
```

So a category-valued family is just a functor into `Cat`. The implementation calls the category of such families `Catd(K)`, but the mathematical reading is:

```
E : K ⊢ Cat.
```

A morphism of families is a natural family of functors:

```
FF : k :n K ; E[k] ⊢ D[k]
FF[k] : E[k] ⊢ D[k].
```

A transformation between such family morphisms is a family of transformations natural in the same directed base:

```
ε : FF ⇒ GG
ε[k] : FF[k] ⇒ GG[k].
```

The notation `:n` marks an ordinary directed index in the current surface syntax. The paper uses typographic operators such as $\rightarrow$, $\rightarrow_*$, $\Rightarrow$, and $\Rightarrow_*$; the parser-oriented ASCII spellings are recorded in the syntax report. Mixed variance appears on the family occurrence, for example `A[z-] ⊢-[z] B[z]`, not by introducing older binder modes.

3. Dependent Hom, Sigma Totals, And Sections

The key construction for v3.2 is the dependent hom over a base arrow. Let

```
E : K ⊢ Cat
x y : K
u : E[x]
v : E[y].
```

For every base arrow $f : x \rightarrow y$, transport sends u to $E[f](u)$ in the fibre over y . The dependent hom over f is then:

$$\mathbf{Hom}_{E[y]}(E[f](u), v).$$

As a whole, this is not only a pointwise formula. It is a `Cat`-valued functor over the base hom:

```
hond_E(x,u,y,v) : Hom_K(x,y)op ⊢ Cat
hond_E(x,u,y,v)[f] = Hom_{E[y]}(E[f](u), v).
```

The `Lambdapi` identifier for this dependent-hom functor is `hond_`. The opposite on the base hom is an implementation convention that makes the functorial action line up with the current normal forms. The reader-facing

content is simply: a dependent arrow from (x, u) to (y, v) consists of a base arrow and a fibre arrow above it.

3.1 Sigma Totals

The dependent sum of a directed family is a total category:

$$\Sigma_k E[k].$$

Its objects are pairs:

$$(k, u) \quad \text{with } k : K \text{ and } u : E[k].$$

The hom category in a total category is organized by the dependent hom. At the level of 1-cell data it has the familiar dependent-pair shape:

$$\begin{aligned} \text{Hom}_{\{\Sigma E\}}((x, u), (y, v)) \\ = \Sigma f : x \rightarrow^K y, \text{Hom}_{\{E[y]\}}(E[f](u), v). \end{aligned}$$

The kernel keeps the opposite orientation needed for higher hom-action explicit, so the checked normal form is an opposite of a Sigma category over the opposite base hom. This distinction affects higher cells and normal forms, not the ordinary description of a total arrow.

Equivalently, an arrow in the total category is a pair:

$$\begin{aligned} (f, \alpha) \\ f : x \rightarrow^K y \\ \alpha : E[f](u) \rightarrow^{E[y]} v. \end{aligned}$$

The implementation identifier for $\Sigma_k E[k]$ is `Sigma_cat(E)`; `sigma_arrow(E, p, α)` names the pair (p, α) . The canonical transported total arrow is `sigma_transport_arrow(E, p, u)`:

$$\text{sigma_transport_arrow}(E, p, u) : (x, u) \rightarrow (y, E[p](u))$$

whose fibre component is the identity at the transported endpoint.

3.2 Pi Sections

The dependent product is the category of sections:

$$\Pi_k E[k].$$

An object $s : \Pi_k E[k]$ assigns a fibre object $s[k] : E[k]$ for every $k : K$, and it also carries action over base arrows:

$$s[f] : \text{Hom}_{\{E[y]\}}(E[f](s[x]), s[y])$$

for $f : x \rightarrow y$. This is the same dependent-hom shape as the Sigma hom, specialized to the section's endpoints. In v3.2 this shared architecture is important: total-category arrows and section actions are not independent features with unrelated computation rules.

The implementation name for the section category is `Pi_cat(E)`. In the surface notation the article writes:

$$\Pi (k : ^n K), E[k].$$

Section evaluation is written $s[k]$ in the surface notation; its Lambdapi projection head is `piapp0(s,k)`.

4. Outgoing Arrows And The Canonical `rho` Arrow

Now fix a category Z and an object $x : Z$. The represented family at x is:

$$\text{Rep}_Z(x)[y] = \text{Hom}_Z(x, y).$$

Its action along a base arrow $p : x \rightarrow y$ is representable precomposition:

$$\begin{aligned} \text{Rep_transport}(p) &: \text{Rep}_Z(y) \rightarrow \text{Rep}_Z(x) \\ \text{Rep_transport}(p)[z][q] &= q \circ p. \end{aligned}$$

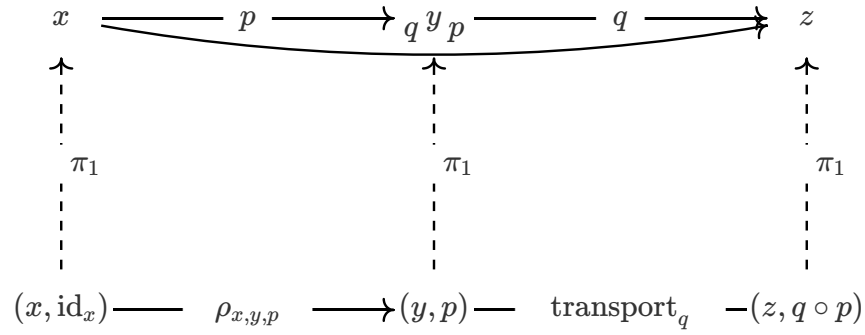
The outgoing-arrow category from x is the Sigma total of this representable:

$$\begin{aligned} \text{PathOut}_Z(x) &= \Sigma y : ^n Z, \text{Rep}_Z(x)[y] \\ &= \Sigma y : ^n Z, \text{Hom}_Z(x, y). \end{aligned}$$

An object of `PathOut_Z(x)` is a pair (y, p) with $p : x \rightarrow^Z y$. The distinguished reflexive outgoing arrow is:

$$\text{reflout}_x = (x, \text{id}_x).$$

The first diagram shows `PathOut_Z(x)` as a total category over Z .



For every object (y, p) of $\text{PathOut}_Z(x)$, there is a canonical arrow:

$$\rho_{\{x,y,p\}} : \text{reflout}_x \rightarrow^{\text{PathOut}_Z(x)} (y, p).$$

This is not postulated. It is the Sigma transport arrow of the representable family:

$$\rho_{\{x,y,p\}} = \text{sigma_transport_arrow}(\text{Rep}_Z(x), p, \text{id}_x).$$

The endpoint computation is:

$$\text{Rep}_Z(x)[p](\text{id}_x) = p.$$

Thus the fibre component of ρ is the identity at the computed endpoint p .

$$\begin{array}{ccc}
 (x, \text{id}_x) & \xrightarrow{\quad \rho_{x,y,p} \quad} & (y, p) \\
 \vdots & & \vdots \\
 \pi_1 & & \pi_1 \\
 \vdots & & \vdots \\
 x & \xrightarrow{\quad p \quad} & y \\
 \vdots & & \vdots \\
 \text{fibre} & & \text{fibre} \\
 \vdots & & \vdots \\
 \text{id}_x & \xrightarrow{\quad \text{Rep}_Z(x)[p] \quad} & p
 \end{array}$$

The `rho` arrows assemble into a section:

```

pathout_refl_arrow_sec(x)
: Π q : ^n PathOut_Z(x),
  reflout_x → ^PathOut_Z(x) q

pathout_refl_arrow_sec(x)[(y,p)] = rho_{x,y,p}.

```

This section is the categorical reason an induction principle is available: every outgoing arrow has a canonical arrow from the reflexive outgoing arrow.

5. Arrow Induction

Let `E : PathOut_Z(x) ⊢ Cat` be a motive over outgoing arrows, and let

```
u : E[reflout_x].
```

Fixed-source arrow induction constructs a section:

```
path_ind_sec(Z,x,E,u) :  $\prod q : ^n \text{PathOut\_Z}(x), E[q]$ .
```

Its computation at an outgoing arrow (y,p) is:

```
path_ind_sec(Z,x,E,u)[(y,p)] = E[rho_{x,y,p}](u).
```

In words: to define an element of the motive at every outgoing arrow, it suffices to define it at the reflexive outgoing arrow and then transport along the canonical rho arrow.

The checked internal version of the same statement uses ordinary fibre transport in the family E . The kernel head `fib_cov_tapp0_func` names the functor $f \mapsto E[f](u)$:

```
piapp0(path_ind_sec(Z,x,E,u),(y,p))
= fib_cov_tapp0_func(PathOut_Z(x),E,reflout_x,(y,p),u)(rho_{x,y,p}).
```

This fixed-source construction is packaged as a shaped functor:

```
PathInd_func(Z,x)
: E : ^n (PathOut_Z(x)  $\vdash$  Cat) ;
  E[(x,id_x)]  $\vdash \prod (q : ^n \text{PathOut\_Z}(x)), E[q]$ 

PathInd_func(Z,x)[E](u) = path_ind_sec(Z,x,E,u).
```

5.1 The Telescope Theorem

The primary v3.2 theorem lets the source object vary. In unfolded surface notation, the theorem is the following source-indexed construction:

```
x : ^n Z ;
E : ^n (PathOut_Z(x)  $\vdash$  Cat) ;
E[(x,id_x)]  $\vdash \prod (q : ^n \text{PathOut\_Z}(x)), E[q]$ .
```

Its value on $u : E[(x, \text{id}_x)]$ is the section whose component at (y,p) is $E[\text{rho}_{x,y,p}](u)$.

The kernel packages this theorem as a transformation between two source-indexed constructions. For each $x : Z$, the motive category is:

```
PathOutMotives_Z[x] = PathOut_Z(x)  $\vdash$  Cat.
```

```

source[x,E] = E[(x,id_x)]
target[x,E] =  $\prod q : ^n \text{PathOut\_Z}(x), E[q]$ .

```

The implementation names for these are `PathOutRefEval_Z` and `PathOutPi_Z`, and the packaged theorem is:

```

PathInd_transfd(Z)
:  $x : ^n Z ; \text{PathOutRefEval\_Z}[x] \Rightarrow \text{PathOutPi\_Z}[x]$ .

```

This packaging is important to the implementation because naturality in the moving source object is part of the theorem's type, not a separate external square.

Along a base arrow $p : x \rightarrow y$, `PathOut` is contravariant in the source. Precomposition gives a functor

```

PathOut_Z(p) : PathOut_Z(y)  $\vdash$  PathOut_Z(x)
PathOut_Z(p)[(z,q :  $y \rightarrow z$ )] = (z,q  $\circ$  p).

```

A motive E over `PathOut_Z(x)` is transported by pullback along this action:

```

p_*E = PathOut_Z(p)^*E : PathOut_Z(y)  $\vdash$  Cat.

```

In `Lambdapi` this is the expression `Pullback_catd(E, PathOut_transport_func(p))`. The source-side transport is concrete:

```

PathIndSrc_transport(p,E) = E[rho_{x,y,p}]
: E[(x,id_x)]  $\vdash$  E[(y,p)].

```

The target-side transport is section pullback:

```

PathIndTgt_transport(p,E)
:  $\prod q : ^n \text{PathOut\_Z}(x), E[q]$ 
 $\vdash \prod q : ^n \text{PathOut\_Z}(y), E[\text{PathOut\_transport}(p)[q]]$ .

```

$$\begin{array}{ccccc}
u \in E[(x, \text{id}_x)] & \xrightarrow{\quad} & E[\rho_{x,y,p}] & \xrightarrow{\quad} & E[\rho_{x,y,p}](u) \in E[(y, p)] \\
\vdots \in & & & & \vdots \in \\
(x, \text{id}_x) & \xrightarrow{\quad} & \rho_{x,y,p} & \xrightarrow{\quad} & (y, p)
\end{array}$$

5.2 The Derived Sigma-Total Presentation

There is also an ordinary total-category presentation of the theorem:

```

PathInd_funcd(Z)
: (x,E) :n Σ x :n Z, (PathOut_Z(x) ⊢ Cat) ;
  E[(x,id_x)] ⊢ Π (q :n PathOut_Z(x)), E[q].

```

It is derived, not primitive:

```

PathInd_funcd(Z) = Sigma_transfd_funcd(PathInd_transfd(Z)).

```

Equivalently, it is the Sigma-totalization of the telescope theorem. Its component at (x, E) reduces back to the object component of the fixed-source functor:

```

PathInd_funcd(Z)[(x,E)] = path_ind_func_fapp0(Z,x,E).

```

This distinction is important. The theorem is naturally telescope-shaped; the Sigma-total form is a useful presentation for ordinary functor-level applications and expanded regression checks.

$$\begin{array}{ccccc}
 E[(x, \text{id}_x)] & \xrightarrow{\quad} & \text{PathInd_transfd}(Z) & \xrightarrow{\quad} & \Pi_q E[q] \\
 \swarrow \text{dashed} & & & \searrow \text{dashed} & \\
 x :^n Z; E : \text{PathOut}_Z(x) \rightarrow \mathbf{Cat} & & & & \\
 \downarrow \text{dotted} & & & & \downarrow \text{dotted} \quad \Sigma \\
 \Sigma \text{ source} & \xrightarrow{\quad} & \text{PathInd_funcd}(Z) & \xrightarrow{\quad} & \Sigma \text{ target}
 \end{array}$$

6. Strictness, Laxity, And Directed Induction

In ordinary dependent type theory and HoTT, substitution is organized around variables and identity/path structure. The computation rules for identity elimination are strict at reflexivity, and many uses of transport are judgemental only after the relevant path has reduced to `refl`.

In emdash v3.2 the base of a family is a directed category. A family $E : K \rightarrow \mathbf{Cat}$ transports fibre objects along actual arrows $p : x \rightarrow y$:

$$E[p](u) : E[y].$$

This directed transport is strict for ordinary functoriality: functor actions are oriented so that composites compute as composites. However, a morphism of families

$$FF : k : ^n K ; E[k] \vdash D[k]$$

does not in general preserve canonical transport arrows strictly. Over $p : x \rightarrow y$ and $u : E[x]$, the two evident endpoints are

$$\begin{aligned} & D[p](FF[x](u)) \\ & FF[y](E[p](u)). \end{aligned}$$

A displayed functor therefore has a directed laxity phenomenon at the component level. We write the associated transport-comparison component as $\chi^{\{FF\}}_{\{p,u\}}$ (read as `cmp(FF,p,u)`), reserving λ for lambda abstraction:

$$\chi^{\{FF\}}_{\{p,u\}} : D[p](FF[x](u)) \rightarrow^{\{D[y]\}} FF[y](E[p](u)).$$

This is useful mathematical notation, but it is not a primitive operation in the active kernel. The computational owner is the internal displayed hom-action. Its capped projection has the form

$$\begin{aligned} & \text{fdapp1_int_hom_fapp0}(FF,p,u,\alpha) \\ & : D[p](FF[x](u)) \rightarrow^{\{D[y]\}} FF[y](v) \end{aligned}$$

for a fibre arrow $\alpha : E[p](u) \rightarrow v$. Thus the Sigma-map action on a total arrow is represented as

$$\Sigma(FF)(p,\alpha) = (p, \text{fdapp1_int_hom_fapp0}(FF,p,u,\alpha)).$$

The familiar composite $FF[y][\alpha] \circ \chi_{p,u}^{FF}$ is only a surface reading of this capped projection. The article does not take it as the definition, because the active v3.2 design deliberately avoids reconstructing Sigma maps from a separate whole-comparison operation.

The canonical identity case extracts the component-level cell:

$$\begin{aligned} & \text{fdapp1_int_hom_fapp0}(FF,p,u,\text{id}_{\{E[p](u)\}}) \\ & = \text{fdapp1_int_cell}(FF,p,u). \end{aligned}$$

This is the sense in which laxity becomes visible: it is observed through the internal displayed hom-action and its Sigma-map fibre component, not supplied as an independent external naturality square.

Strict or cartesian constructors add focused computation rules making the comparison cell collapse. Representable precomposition is one such strict case: for $p : x \rightarrow y$, the family morphism

$$\text{Rep_transport}(p) : \text{Rep_Z}(y) \vdash \text{Rep_Z}(x)$$

has cartesian behaviour on the canonical identity fibre arrow. Concretely, for $q : y \rightarrow z$, the comparison component reduces to the identity at the composite:

$$\chi^{\{\text{Rep_transport}(p)\}}_{\{q,\text{id}_y\}} = \text{id}_{\{q \circ p\}}.$$

This is why the `PathOut` and composition benchmarks compute strictly even though the surrounding directed-family theory permits lax displayed transport.

The formulation of arrow induction follows this discipline. The primary theorem is the source-indexed telescope:

```
x : ^n Z ;
E : ^n (PathOut_Z(x) ⊢ Cat) ;
E[(x, id_x)] ⊢ Π (q : ^n PathOut_Z(x)), E[q].
```

Moving the source object along $p : x \rightarrow y$ transports motives by pullback along `PathOut_Z(p)`. The source side is the concrete map $E[\rho_{x,y,p}]$, and the target side is section pullback. The derived Sigma-total theorem is used to inspect this structure at canonical transported objects. It should not be read as a claim that arbitrary non-cartesian Sigma-total arrows preserve the induction structure strictly.

7. Composition As The Main Computation

The most concrete application of arrow induction is directed transitivity. Fix $x : Z$. Define the composition target family:

```
CompTarget_Z(x)[y] = Rep_Z(y) ⊢ Rep_Z(x).
```

Pull it back along the projection `PathOut_Z(x) → Z`:

```
CompMotive_Z(x)[(y,p)] = Rep_Z(y) ⊢ Rep_Z(x).
```

Path induction at the identity displayed functor gives:

```
path_comp_sec(x)
= path_ind_sec(CompMotive_Z(x), id_Rep_Z(x)).
```

Its first application to $p : x \rightarrow y$ is a functor:

```
path_comp_func(p) : Rep_Z(y) ⊢ Rep_Z(x).
```

Its action on $q : y \rightarrow z$ computes to ordinary composition:

```
path_comp_func(p)[z][q] = q ∘ p.
```

The current runtime normal form for this composite is the hom-action cut:

```
hom_postcomp_fapp0 Z Z (id_func Z) x y z q p.
```


The ordinary categorical composite is still present as the proof-time view `comp_fapp0 Z x y z q p`. The distinction is intentional: the telescope `hom_postcomp_*` and `hom_precomp_along_*` heads own runtime cut-elimination, while `comp_fapp0`, `comp_cat_fapp0`, and `comp_catd_fapp0` remain the ordinary composition and convertibility surfaces.

$$\begin{array}{c}
 p : x \rightarrow y \cdots \cdots \text{input} \cdots \cdots q : y \rightarrow z \text{---} \circ \text{---} q \circ p : x \rightarrow z \\
 \vdots \qquad \qquad \qquad \vdots \qquad \qquad \qquad \vdots \\
 \text{CompMotive}_Z(x) \quad \text{id}_{\text{Rep}_Z(x)} \quad \text{---PathInd---} \quad \sim \text{hom_postcomp_fapp0}(\text{id}_Z, q, p)
 \end{array}$$

Both theorem presentations reach this normal form. The primary telescope route is checked as:

```

PathInd_transfd(Z)[x][CompMotive_Z(x)](id_Rep_Z(x))[p][z][q]
= hom_postcomp_fapp0 Z Z (id_func Z) x y z q p.

```

The derived Sigma-total route is checked as:

```

PathInd_funcd(Z)[(x, CompMotive_Z(x))](id_Rep_Z(x))[p][z][q]
= hom_postcomp_fapp0 Z Z (id_func Z) x y z q p.

```

The computation is not a special-purpose rewrite for transitivity. It is the ordinary action of the representable family, reached by applying the general arrow-induction theorem to the composition motive.

8. Surface Syntax And Kernel Names

The paper uses the current v3.2 surface syntax. The syntax distinguishes ordinary homs, indexed homs, ordinary functor categories, shaped functor categories, section categories, and displayed transformation categories.

$a \multimap^C b$	$\text{Hom_cat } C \ a \ b$	ordinary hom with explicit ambient category
$a \rightarrow b$	$\text{Hom_cat } C \ a \ b$	ordinary hom when C is clear
$aa[z^\wedge -] \rightarrow_{[z]}^\wedge bb[z]$	$\text{Hom_catd } R \ aa \ bb$	indexed/displayed hom
$A \vdash B$	$\text{Functor_cat } A \ B$	ordinary functor/program category
$z :^\wedge n \ Z ; E[z] \vdash D[z]$	$\text{Functord_cat } E \ D$	shaped functor/program category
$\Pi (z :^\wedge n \ Z), D[z]$	$\text{Pi_cat } D$	terminal-shape section category
$A[z^\wedge -] \vdash_{[z]} B[z]$	$\text{Functor_catd } A \ B$	mixed-variance displayed functor family
$F \Rightarrow G$	$\text{Transf_cat } F \ G$	ordinary transformation category
$FF[z^\wedge -] \Rightarrow_{[z]} GG[z]$	$\text{Transf_catd } A \ B \ FF \ GG$	indexed/displayed transformation category

Two notation choices prevent ambiguity.

First, ordinary ambient categories use superscripts:

$a \multimap^C b.$

The operator $\rightarrow_{-}$ is reserved for indexed or displayed homs:

$aa[z^\wedge -] \rightarrow_{[z]}^\wedge bb[z].$

Second, the shaped category

$z :^\wedge n \ Z ; E[z] \vdash D[z]$

does not bind an object variable of $E[z]$. The shape $E[z]$ is part of the generalized quantification. If the target depends on an actual object of $E[z]$, the construction is a Sigma-style telescope.

The nested telescope stress test is:

$k :^\wedge n \ K ; C[k] \vdash (z :^\wedge n \ Z ; E[k^\wedge -; z] \vdash D[k; z]).$

This is telescope-style notation, not product-base notation. The order $k; z$ is meaningful.

9. Computational Method And Checked Evidence

The formalization relies on a small normalization discipline. Constructions whose projections are used by later theorems are assigned stable semantic owners; readable helper names are routed through those owners rather than

For example:

```
CompTarget_Z(x)[y] = Rep_Z(y) ⊢ Rep_Z(x).
```

The helper for its action on a base arrow routes through `CompTarget_Z(x)` ; it is not a second definition of the same operation.

The stable heads that appear in the evidence have direct mathematical readings:

```
fapp0          ordinary functor object action
fappl_fapp0    ordinary functor arrow action
tapp0_fapp0    transformation component
tdapp0_fapp0   displayed-transformation component
piapp0         section evaluation
```

Rules are installed at the lowest stable head that carries the relevant discriminator. This is why diagnostic assertions often use canonical forms such as `Hom_cat` , `Functor_cat` , and `Functord_cat` rather than readability wrappers.

The formal artifact separates definitions from executable claims: `emdash3_2.lp` contains the definitions and rewrite rules, while `emdash3_2_checks.lp` contains the conversion assertions used as regression evidence.

The surrounding artifact is deliberately modest. The foundations and notation reports explain the mathematical and surface-syntax readings; the current SOP records rewrite-rule hygiene and diagnostic practice; the print pipeline renders this article and validates embedded diagrams. The standard checks typecheck the active `Lambdapi` files under a bounded timeout, run reviewer examples and catalog freshness checks, lint active references and rewrite-rule left-hand sides, and smoke-test the paper renderer. The Infinity Codex archive records final-response recovery evidence after interruptions, but it is not a mathematical authority; active code, reports, and executable checks remain the evidence base.

Representative checked claims:

Surface	Representative checked claim
Representable	$\text{Rep_Z}(x)[y] = \text{Hom_Z}(x, y)$
PathOut object layer	$\text{PathOut_Z}(x) = \text{Sigma_cat } Z (\text{Rep_Z}(x))$
PathOut transport	$\text{PathOut_Z}(p) : \text{PathOut_Z}(y) \vdash$ $\text{PathOut_Z}(x) \text{ and } \text{PathOut_Z}(p)[(z, q : y \rightarrow z)] = (z, q \circ p)$
Rho construction	$\text{rho}\{x, y, p\} =$ $\text{sigma_transport_arrow}(\text{Rep_Z}(x), p, \text{id}_x)$
Fixed-source induction	$\text{PathInd_func}(Z, x)[E](u) =$ $\text{path_ind_sec}(Z, x, E, u)$
Telescope theorem	$\text{PathInd_transfd}(Z)[x] =$ $\text{PathInd_func}(Z, x)$
Sigma-total theorem	$\text{PathInd_funcd}(Z)[(x, E)] =$ $\text{path_ind_func_fapp0}(Z, x, E)$
Composition benchmark	$\text{path_comp_func}(p)[z][q] = q \circ p$

10. Cat-Valued Profunctors And Representables

The second arc of the v3.2 development starts from Cat-valued profunctors. For categories A and B , emdash defines the profunctor category by the same category-valued-family infrastructure used earlier:

```

Prof(A,B)      = A^op × B ⊢ Cat
Prof_cat(A,B) = Catd_cat(Product_cat(Op_cat A) B)
ProfMap(P,Q)  = Obj (Hom_{Prof_cat(A,B)}(P,Q)).

```

Thus a profunctor is not a new external semantic primitive. It is a directed family over the product base $A^{\text{op}} \times B$, and a vertical profunctor map is an object of the ordinary hom-category between two such families, equivalently a displayed transformation in that family category. This choice keeps identity and vertical composition owned by the existing `id_funcd` and `comp_catd_fapp0` calculus.

The representable profunctor associated to two functors $F : A \vdash X$ and $G : B \vdash X$ is written:

```

Hom_prof_along(F,G) : Prof(A,B)
Hom_prof_along(F,G)[a,b] = Hom_X(F[a],G[b]).

```

Its action on an arrow pair $(p : a' \rightarrow a, q : b \rightarrow b')$ is the expected cut:

```

h : Hom_X(F[a],G[b])
-----
Hom_prof_along(F,G)[p,q](h) = G[q] ∘ h ∘ F[p].

```

The implementation deliberately routes this computation through the generic hom-action owners. `hom_postcomp_*`, `hom_precomp_along_*`, and their `fapp0` projections expose the reusable runtime cuts; `comp_fapp0`, `comp_cat_fapp0`, and `comp_catd_fapp0` remain the ordinary composition heads and proof-time comparison surfaces.

Endpoint reindexing is also internal:

```

Prof_reindex(R,F,G)[a',b'] = R[F[a'],G[b']]

```

with a functorial owner

```

Prof_reindex_func(F,G) : Prof(A,B) ⊢ Prof(A',B').

```

The checks cover identity endpoint reindexing, nested endpoint reindexing, representable reindexing, and the semantic agreement with pullback along the product map. This is the profunctor version of the earlier displayed-family discipline: a readable operation is allowed, but it must route through one stable owner.

Two representables receive standard equipment names:

```

Companion_prof(F) = Hom_prof_along(F,id)
Conjoint_prof(F)  = Hom_prof(F).

```

The internal right-representable embedding

```

Hom_prof_func(J,B) : (J ⊢ B) ⊢ Prof(B,J)

```

computes on objects to `Hom_prof(G)`. Its body is semantic, built from ordinary precomposition, internal hom, and uncurrying, but the public head is opaque so that projection rules do not repeatedly unfold the composite.

The shaped-cell layer is the bridge from vertical profunctor maps to equipment notation. A cell over endpoint functors $F : A' \vdash A$ and $G : B' \vdash B$ is:

```

Prof_transf_cat(R',F,R,G)
= R' ⊢ Prof_reindex(R,F,G).

```

A shaped element is the special case whose source is the unit profunctor on a shape category I :

```

Prof_hom(I,A,B,F,R,G)
= Obj(Prof_transf_cat(Unit_prof(I),F,R,G)).

```

The narrow evaluator `Prof_cell_apply` applies one internally natural profunctor cell to one shaped element. It is intentionally smaller than a full bicategory-of-profunctors composition operator. Its role is to support the checked co-Yoneda and join computations below without adding broad constructor-local functoriality rules.

11. Tensor, Co-Yoneda, And Internal Hom

The active tensor is a symbolic profunctor composite:

```
P : Prof(A,B), Q : Prof(B,X)
-----
Prof_tensor(P,Q) : Prof(A,X).
```

Mathematically this is the coend-like composite of profunctors. The current kernel does not yet contain a general coend or coinsertion quotient, so `Prof_tensor` is intentionally opaque. Only the endpoint behavior that is needed by checked consumers is computational: reindexing distributes over the two exposed endpoints, while the shared middle category remains fixed.

Fixed-endpoint tensor functoriality is internalized by a bifunctor:

```
Prof_tensor_func(A,B,X)
: Prof(A,B) × Prof(B,X) ⊢ Prof(A,X)
```

with projections:

```
Prof_tensor_func(P,Q) = Prof_tensor(P,Q)
Prof_tensor_func(r,s) = Prof_tensor_map(r,s).
```

This is the current owner for fixed tensor action. The broader endpoint-changing tensor-transformer names from earlier experiments are not active core owners; future endpoint-changing tensor theorems should be derived from reindexing plus this fixed-endpoint map layer once the missing middle-change/coend compatibility is available.

The co-Yoneda maps are fixed natural transformations from tensoring with the unit profunctor back to the identity:

```
Prof_coyoneda_cov_transf(A,B)
: Prof_tensor_right_unit_func(A,B) ⇒ id_{Prof(A,B)}

Prof_coyoneda_con_transf(A,B)
: Prof_tensor_left_unit_func(A,B) ⇒ id_{Prof(A,B)}.
```

Their diagonal components are stable projection heads:

```
Prof_coyoneda_cov_map(P)
: ProfMap(Prof_tensor(P,Unit_prof(B)), P)

Prof_coyoneda_con_map(P)
: ProfMap(Prof_tensor(Unit_prof(A),P), P).
```

At shaped elements, the checked beta rules have the expected co-Yoneda shape. For $p : \text{Prof_hom}(I, A, B, F, P, M)$:

```
Prof_cell_apply(
  Prof_coyoneda_cov_map(P),
  Prof_tensor_hom_hom(M,p,Prof_func_hom(M)))
= p.
```

Dually, for $p : \text{Prof_hom}(I, A, B, F, P, G)$:

```
Prof_cell_apply(
  Prof_coyoneda_con_map(P),
  Prof_tensor_hom_hom(F,Prof_func_hom(F),p))
= p.
```

Naturality fusion is also checked for arbitrary fixed-endpoint maps. Applying the `tapp1_fapp0` component of a co-Yoneda transformation to a tensor-shaped element reduces to applying the chosen map to the original shaped element. This is the Došen-style reading of co-Yoneda in the kernel: cut with the co-Yoneda counit eliminates.

The closed structure is represented by covariant and contravariant implication cores:

```
Prof_imply_cov(0,Q) : Prof(A,B)
Prof_imply_con(P,0) : Prof(B,X)
```

They are paired with evaluation and lambda maps:

```
Prof_eval_cov_map
Prof_lambda_cov_map
Prof_eval_con_map
Prof_lambda_con_map.
```

For the covariant side, maps

```
ProfMap(Prof_tensor(P,Q), 0)
```

and

```
ProfMap(P, Prof_imply_cov(0,Q))
```

convert back and forth by computation. The contravariant side has the dual pair of conversions. Additional shaped and endpoint-changing views exist where concrete consumers need them, but the principal article point is simpler: tensor, co-Yoneda, and internal hom are not detached axioms. They are exposed as a cut-elimination interface over Cat-valued profunctors.

12. Weighted Limits, Adjunctions, And Duality

Weighted limits are formulated as profunctor representability. For $F : J \vdash B$ and a weight $W : \text{Prof}(J', J)$, the weighted-cone profunctor is:

```
WeightedCone_prof(F,W)
= Prof_imply_cov(Hom_prof(F), W).
```

A covariant weighted limit of F by W at $L : J' \vdash B$ is a computational comparison:

```
WeightedLimit_cov(F,W,L)
= ProfComparison(WeightedCone_prof(F,W), Hom_prof(L)).
```

There is also an ordinary `IsoEvidence` presentation, `IsWeightedLimit_cov_iso`. The active unsuffixed public name is the stronger computational comparison, and the ordinary evidence is obtained by projection. This is a recurring emdash pattern: a proposition-like statement may be useful for mathematical reading, but the runtime theorem is the pair of operations that eliminate against arbitrary probes.

Concretely, for any incoming profunctor R and any shaped probe $M : I \vdash B$, the comparison supplies inverse operations:

```
weighted_limit_cov_push
weighted_limit_cov_pull
```

and the checks assert:

```
pull(push(r)) = r
push(pull(s)) = s.
```

Selected universal and cone maps are exposed as identity applications of these operations:

```
weighted_limit_cov_univ_transf
weighted_limit_cov_cone_transf.
```

The preservation theorem for right adjoints is then an actual computation over these comparisons. Given an adjunction between A and B , the representable mate comparison is:

```
Adjunction_hom_prof_comparison(adj)
: Hom_B(left(-),-) ≈ Hom_A(-,right(-)).
```

The theorem


```
right_adjoint_preserves_weighted_limit_cov
```

is transparently the composition of three comparison steps:

1. use the inverse adjunction mate through fixed-weight implication;
2. reindex the given weighted-limit comparison along the left adjoint;
3. use the adjunction mate at the candidate limit.

The ordinary theorem `right_adjoint_preserves_weighted_limit_cov_iso` is defined in parallel over `IsoEvidence`, and the computational comparison forgets exactly to this ordinary evidence. The formal checks include that compatibility.

Duality is handled by opposite normalization rather than by duplicating the proof. The profunctor duality operation

```
Op_prof(P) : Prof(Op B, Op A)
```

pulls back along product swap and does not opposite the fibre categories. This choice is deliberate: with the current hom rule for opposite categories, pointwise fibre-op would dualize representable hom categories twice.

Contravariant weighted colimits are transparent weighted limits in the opposite world:

```
WeightedColimit_con(F,W,L)  
= WeightedLimit_cov(Op_func(F), Op_prof(W), Op_func(L)).
```

The two conversion wrappers `Op_weighted_limit_cov` and `Op_weighted_colimit_con` are identities after opposite and double-swap normalization. Consequently,

```
left_adjoint_preserves_weighted_colimit_con
```

is obtained by applying right-adjoint preservation to `Op_adjunction`. No second colimit-specific cut calculus is installed.

13. Directed-Inductive Join Categories

The first directed-inductive stress test is a primitive join category. For categories A and B , the kernel has:

```
Join_cat(A,B)  
join_fst_func : A ⊢ Join_cat(A,B)  
join_snd_func : B ⊢ Join_cat(A,B).
```

The directed cross arrows from the A side to the B side are not represented as an externally quantified family plus a separate naturality proof. They are one internally natural profunctor cell:

```

join_cross_transf
  : Terminal_prof(A,B)
    ⊢ Hom_prof_along(join_fst_func, join_snd_func).

```

The old shaped cross-arrow interface is derived from that cell:

```

join_cross_hom(a,b)
  = Prof_cell_eval(join_cross_transf,a,b).

```

The nondependent recursor is:

```

join_elim_func(first,second,cross) : Join_cat(A,B) ⊢ E.

```

It computes on both inclusions:

```

join_elim_func(first,second,cross) ◦ join_fst_func = first
join_elim_func(first,second,cross) ◦ join_snd_func = second

```

and on the cross cell:

```

join_elim_cross_transf(first,second,cross) = cross.

```

This is not yet a semantic collage construction and not yet a generic dependent higher-inductive schema. Its purpose in v3.2 is narrower and useful: it tests whether directed-inductive data can be expressed using the same profunctor-cell, shaped-element, and cut-elimination interfaces developed for the tensor and weighted-limit calculus.

14. Equality, DefIso, And Normalization Boundaries

The v3.2 kernel now contains several equality-like layers, and the article keeps them separate.

The ordinary categorical path/equality layer supports the internal syntax needed by the directed-family development. Groupoid and univalence-facing interfaces stage equivalence data:

```

PathOver
TypeEquiv
GrpdUnivalence
OmegaEquiv
CatUnivalence.

```

These are not advertised here as a completed semantics of all categories. They are a computational interface for later univalence and higher-categorical work.

The strict computational isomorphism layer is `DefIso`. It packages selected forward and backward maps together with cancellation behavior that is meant to compute. Public profunctor comparisons are now compatibility names:

```
ProfComparison(P,Q) = DefIso(Prof_cat(A,B),P,Q).
```

Their eliminators are ordinary hom-action cuts:

```
prof_comparison_push(i,r)
  = hom_postcomp_fapp0(id, defiso_to(i), r)

prof_comparison_pull(i,s)
  = hom_postcomp_fapp0(id, defiso_from(i), s).
```

The cancellation equations for comparison push and pull are therefore inherited from `DefIso` and `hom_postcomp_fapp0`, not from theorem-specific rewrite rules. This is the same architectural boundary already visible in the PathOut composition benchmark:

```
hom_postcomp_* / hom_precomp_along_* / hom_postcomp_fapp0
runtime cut-elimination owners

comp_fapp0 / comp_cat_fapp0 / comp_catd_fapp0
ordinary composition heads and proof-time comparison surfaces.
```

The boundary also explains the treatment of Cat-specialized heads. A specialization such as a `Cat_cat` or `Catd_cat` hom-action projection is kept when it exposes data that is only semantically available at that level, for example `tapp0_fapp0`, `tapp1_func`, or `tapp1_fapp0`. It is not a license to add a second owner for ordinary functoriality or naturality.

The current limitations are part of the formal claim.

- `Prof_tensor` has no general coend or coinserter semantics yet; only the fixed endpoint and co-Yoneda-facing computations are active.
- `Join_cat` is a primitive directed-inductive stress test, not a constructed semantic collage.
- Generic dependent join elimination remains future work.
- Full bicategory/equipment coherence is not claimed.
- General dependent adjunctions such as $\Sigma_F \dashv F^* \dashv \Pi_F$ remain future work.
- Arbitrary Sigma-map transport is not claimed to be strict without the corresponding strict or cartesian specialization.
- The project is still a strict/lax computational core for higher-categorical programming, not a completed formalization of weak ω -categories.

15. Conclusion

The v3.2 development shows that directed arrow induction can be expressed as computational categorical structure. The foundation is a directed dependent type theory of category-valued families, Sigma totals, Pi sections, and dependent homs. The outgoing-arrow category is a Sigma total of a representable; the canonical `rho` arrow is Sigma transport; and the primary theorem is a source-indexed telescope from the reflexive fibre of a motive to its section category.

The composition benchmark is the visible payoff. Applying arrow induction to the composition motive normalizes to ordinary categorical composition, with `hom_postcomp_fapp0` as the runtime cut-elimination owner and

`comp_fapp0` as the typed ordinary-composition view.

The newer profunctor layer shows that the same method scales beyond the PathOut theorem. Cat-valued profunctors, tensor, co-Yoneda reductions, internal homs, weighted limits, adjunction preservation, dual colimits, and a primitive join category all use the same discipline: put the categorical operation behind a stable owner, expose the projections needed by downstream consumers, and make structural coherence compute when it is meant to be a normal form. That is the emdash design goal in miniature: coherence is not merely stated; it computes.

Appendix A. Identifier Glossary

Mathematical notation	Kernel identifier
category of categories	Cat_cat
$E : K \vdash \text{Cat}$	Catd_cat K
fibre $E[k]$	Fibre_cat K E k
transport $E[p]$	catd_transport_func K E x y p
$A \vdash B$	Functor_cat A B
$z : ^n Z ; E[z] \vdash D[z]$	Functord_cat Z E D
$\prod (k : ^n K), E[k]$	Pi_cat K E
section evaluation $s[k]$	piapp0 K E s k
$\Sigma (k : ^n K), E[k]$	Sigma_cat K E
total arrow (p, α) in ΣE	sigma_arrow K E x y u v p α
canonical total transport	sigma_transport_arrow K E x y p u
Sigma map $\Sigma(FF)$	sigma_map_func K E D FF
dependent hom $\text{homd}_E(x, u, y, v)$	homd_ K E E (id_funcd K E) x u y v
motive pullback F^*E	Pullback_catd A B E F
uncurrying a displayed theorem over Sigma	Sigma_transfd_funcd K R S T η
$\text{Rep}_Z(x)$	Rep_catd Z x
$\text{Rep_transport}(p)$	Rep_transport_func Z x y p
$\text{PathOut}_Z(x)$	PathOut_cat Z x
$\text{PathOut}_Z(p)$	PathOut_transport_func Z x y p
(y, p) in $\text{PathOut}_Z(x)$	pathout_obj Z x y p
(x, id_x)	pathout_refl_obj Z x
$\text{rho}_{\{x, y, p\}}$	pathout_refl_arrow Z x y p
section of all rho arrows	pathout_refl_arrow_sec Z x
$E[\text{rho}_{\{x, y, p\}}]$	pathout_refl_eval_base_func Z x y p E
transported motive p_*E	pathout_motive_transport_obj Z x y p E
PathOutMotives_Z	PathOutMotives_catd Z
PathOutRefLEval_Z	PathOutRefLEval_funcd Z

PathOutPi_Z	PathOutPi_funcd Z
fixed-source induction source family	PathInd_src_catd Z x
fixed-source induction target family	PathInd_tgt_catd Z x
fixed-source component functor	path_ind_func_fapp0 Z x E
fixed-source path induction	PathInd_func Z x
telescope path induction	PathInd_transfd Z
Sigma-total source family	PathIndSrc_catd Z
Sigma-total target family	PathIndTgt_catd Z
Sigma-total source transport	PathIndSrc_transport_func Z x y p E
Sigma-total target transport	PathIndTgt_transport_func Z x y p E
Sigma-total path induction	PathInd_funcd Z
composition target	CompTarget_catd Z x
action of composition target	CompTarget_fapp1_func Z x a b p
composition motive	CompMotive_catd Z x
composition section	path_comp_sec Z x
composition functor for p	path_comp_func Z x y p
fibre transport action $E[f](u)$	fib_cov_tapp0_func K E x y u
covariant fibre transport	fib_cov_transf Z D x u
Sigma-map fibre component	fdapp1_int_hom_fapp0 K E D FF x y p u v α
displayed transport-comparison component $\chi^{\{FF\}}_{\{p, u\}}$	fdapp1_int_cell K E D FF x y p u
profunctor base $A^{\text{op}} \times B$	Prof_base A B
profunctor category $\text{Prof}(A, B)$	Prof_cat A B
profunctor object sort	Prof A B
profunctor map $P \Rightarrow Q$	ProfMap A B P Q
unit profunctor	Unit_prof B
representable profunctor $\text{Hom}_X(F[-], G[-])$	Hom_prof_along A B X F G
conjoint/right representable	Hom_prof B X G
right-representable embedding	Hom_prof_func J B
endpoint reindexing	Prof_reindex A A' B B' R F G

profunctor shaped cell category	Prof_transf_cat A A' B B' R' F R G
profunctor shaped element	Prof_hom I A B F R G
apply shaped profunctor cell	Prof_cell_apply ... c r
tensor of profunctors	Prof_tensor A B X P Q
fixed-endpoint tensor bifunctor	Prof_tensor_func A B X
fixed-endpoint tensor map	Prof_tensor_map A B X P P' Q Q' r s
tensor shaped composition	Prof_tensor_hom_hom I A B X R S F G M r s
covariant co-Yoneda map	Prof_coyoneda_cov_map A B P
contravariant co-Yoneda map	Prof_coyoneda_con_map A B P
covariant internal hom	Prof_imply_cov A B X O Q
contravariant internal hom	Prof_imply_con A B X P O
covariant eval/lambda maps	Prof_eval_cov_map , Prof_lambda_cov_map
contravariant eval/lambda maps	Prof_eval_con_map , Prof_lambda_con_map
weighted cone profunctor	WeightedCone_prof B J J' F W
weighted limit comparison	WeightedLimit_cov B J J' F W L
weighted limit push/pull	weighted_limit_cov_push , weighted_limit_cov_pull
adjunction hom comparison	Adjunction_hom_prof_comparison A B adj
right adjoints preserve weighted limits	right_adjoint_preserves_weighted_limit _cov
profunctor opposite	Op_prof A B P
weighted colimit by duality	WeightedColimit_con B J J' F W L
left adjoints preserve weighted colimits	left_adjoint_preserves_weighted_colimi t_con
directed join category	Join_cat A B
join inclusions	join_fst_func A B , join_snd_func A B
join cross cell	join_cross_transf A B
join recursor	join_elim_func A B E first second cross
strict computational isomorphism	DefIso C x y

profunctor comparison compatibility	ProfComparison A B P Q
comparison push/pull	prof_comparison_push , prof_comparison_pull